JOHANNES KEPLER
UNIVERSITY LINZ

Logic Synthesis and the Standard-Cell Library



Simon Dorrer

KV VLSI Design, 336.051, WiSe 26/27

INSTITUTE FOR
INTEGRATED CIRCUITS &
QUANTUM COMPUTING

JOHANNES KEPLER
UNIVERSITÄT LINZ
Altenberger Straße 69
4040 Linz, Österreich
jku.at

Today's Goals

After this lecture, you can

- name the **steps of synthesis** and the Yosys commands behind them
- explain **standard cells**, rows, and the **views** of a library
- read a **Liberty file**: function, area, timing arcs, tables
- compute a **cell delay** by interpolation, and check it with OpenSTA
- pick the **corner** for a timing check
- trade **area against speed** in technology mapping
- prove a netlist **equivalent** to its RTL with **EQY**



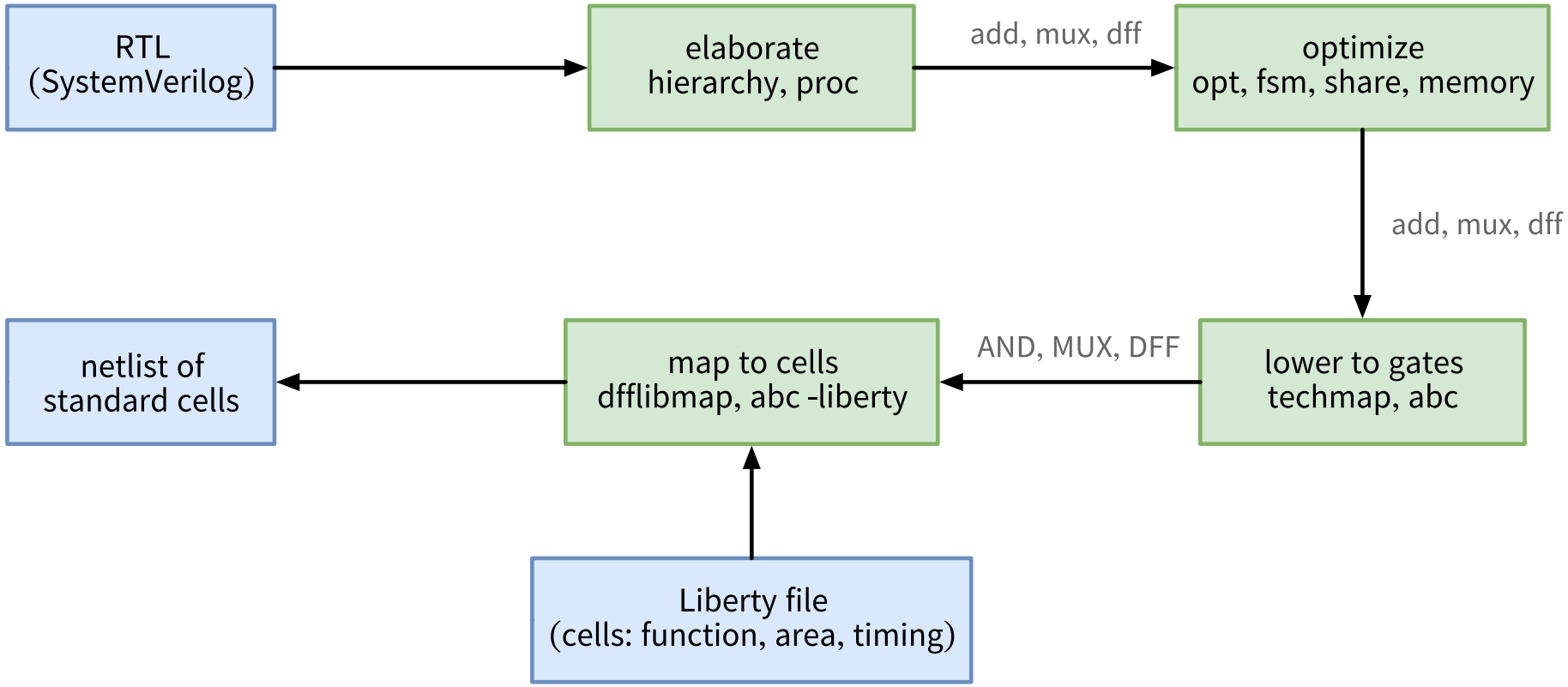
Chapter 5

Lecture notes: Chapter 5 of iic-jku.github.io/vlsi-design

What Logic Synthesis Does



From RTL to Standard Cells



Only the **technology mapping** reads the Liberty file

Generic Gates, Then Cells

```
yosys -p "read_verilog -sv rtl/mux4.sv; synth -top mux4; stat"
yosys -p "read_verilog -sv rtl/mux4.sv; synth -top mux4; \
  dfflibmap -liberty $LIB; abc -liberty $LIB; opt_clean; \
  stat -liberty $LIB"
```

`$LIB`: the Liberty file of the typical corner, defined at the start of Chapter 5

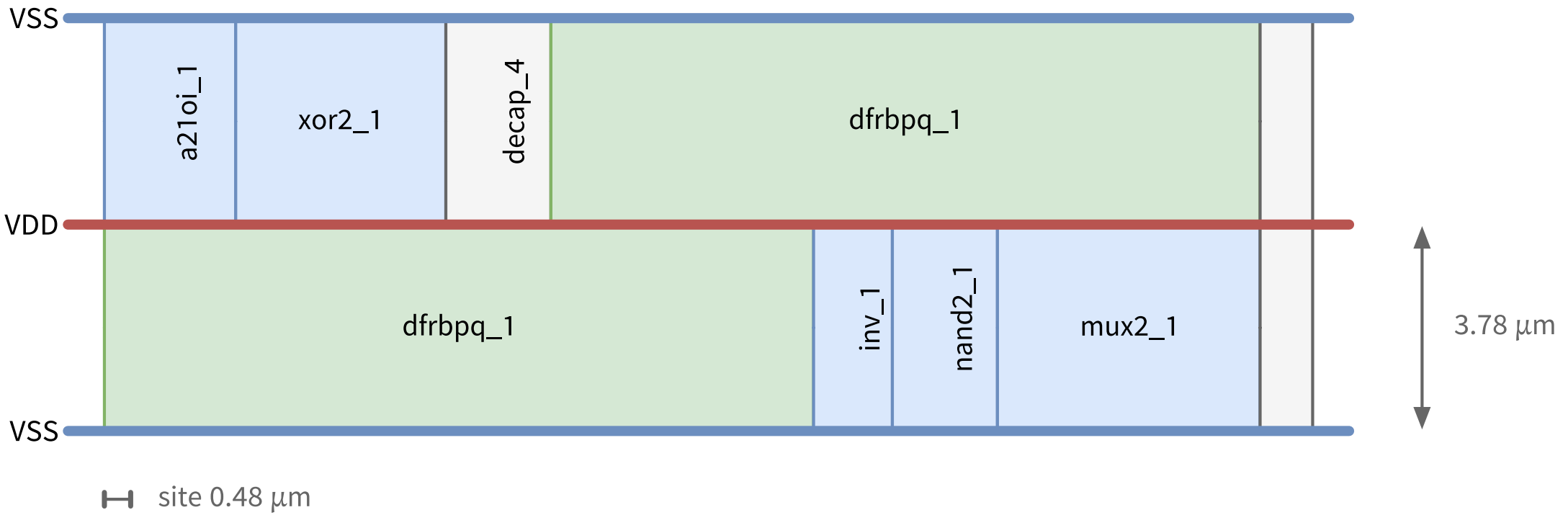
mux4	After synth	After the mapping
cells	67	11
types	40 NAND, 20 AND, 5 NOR, 2 AND-NOT	8 mux4_1, 2 nor4_1, 1 and2_1
area	no library cells yet	335.7 μm²

- `synth` runs the stages **begin, coarse, fine, check**
- ABC found the **4:1 multiplexer cell** of the library

The Standard-Cell Library



Standard Cells in Rows



- Same height **3.78 µm**, width a multiple of the **0.48 µm site**
- Cells **abut** and share the **VDD and VSS rails**, every second row **flipped**

84 Cells in SG13CMOS5L

Category	Cells	Examples
combinational logic	49	inv, buf, nand2, nor3, a21oi, xor2, mux2, mux4
flip-flops	9	dfrbpq, dfrbp, scan: sdfrbpq, sdfbbp, all with reset
latches	5	dlhq, dllrq
clock gates	2	lgcp, slgcp
tri-state drivers	6	ebufn, einvn
delay cells	3	dlygate4sd1 to dlygate4sd3, for hold fixing
tie cells	2	tiehi, tielo
physical cells	7	fill_1 to fill_8, decap_4, decap_8, antennanp
bus holder	1	sighold

- Names: `sg13cmos5l_<function>_<drive>`, e.g., `nand2_1`, `inv_4`

One Cell, Many Views

View	Content	Used by
Liberty	function, area, pin capacitances, delay, power, per corner	synthesis, STA, LEC
LEF	size, pin shapes, blockages, the site, the layers	placement and routing
GDS	the full layout	chip layout, DRC, LVS
Verilog	simulation models with specify blocks	gate-level simulation
CDL, SPICE	transistor netlist	LVS, analog simulation
datasheet	function, pins, timing, one PDF per corner	the designer

- All in `$PDK_ROOT/ihp-sg13cmos5l/libs.ref/sg13cmos5l_stdcell`

Gate Equivalents

Cell	Area	GE
nand2_1	7.2576 µm²	1
inv_1	5.4432 µm²	0.75
mux4_1	38.1024 µm²	5.25
dfrbpq_1	48.9888 µm²	6.75

- **GE** = area / area of a 2-input NAND
- FIFO of Chapter 4: 103 cells, 3008 µm² = **414.5 GE**
- Compares across technologies, but the **ratios differ** between libraries

Inside a Liberty File



The Library Header

```
library (sg13cmos5l_stdcell_typ_1p20V_25C) {
  delay_model : table_lookup;
  capacitive_load_unit (1,pf);
  leakage_power_unit : "1pW";
  time_unit : "1ns";
  ...
  input_threshold_pct_rise : 50;
  nom_process : 1;
  nom_temperature : 25;
  nom_voltage : 1.2;
  output_threshold_pct_rise : 50;
  slew_lower_threshold_pct_rise : 20;
  slew_upper_threshold_pct_rise : 80;
  ...
```

- Plain text from Synopsys, the **de-facto standard**
- Times in **ns**, loads in **pF**
- **Delay**: 50 % in to 50 % out
- **Slew**: 20 % to 80 % of the swing
- One file per **corner**

A Cell: inv_1

```
cell (sg13cmos5l_inv_1) {
  area : 5.4432;
  cell_leakage_power : 63.0032;
  pin (Y) {
    direction : "output";
    function : "!(A)";
    max_capacitance : 0.3;
    timing () {
      related_pin : "A";
      timing_sense : negative_unate;
      timing_type : combinational;
      cell_rise (TIMING_DELAY_7x7ds1) { ... }
      /* rise_transition, cell_fall, fall_transition */
    }
  }
  pin (A) {
    direction : "input";
    capacitance : 0.00286745;
  }
}
```

- `function`: for mapping and LEC
- `capacitance`: the load on the driver, **2.87 fF**
- `max_capacitance`: largest load, **0.3 pF**
- One **timing arc** per input-output pair
- Four **7×7 tables**: delay and slew, rising and falling

Timing Arcs and Unateness

timing_sense	Input rises, output	Examples
positive_unate	rises	AND, OR, buffer
negative_unate	falls	NAND, NOR, inverter
non_unate	rises or falls	XOR, select of a mux, clock of a flip-flop

- `timing_type` says **what kind** of arc it is
  - `combinational`, `rising_edge` (clock to Q)
  - `setup_rising`, `hold_rising`: timing checks of D
  - `recovery_rising`, `removal_rising`: the asynchronous reset

Flip-Flops in Liberty

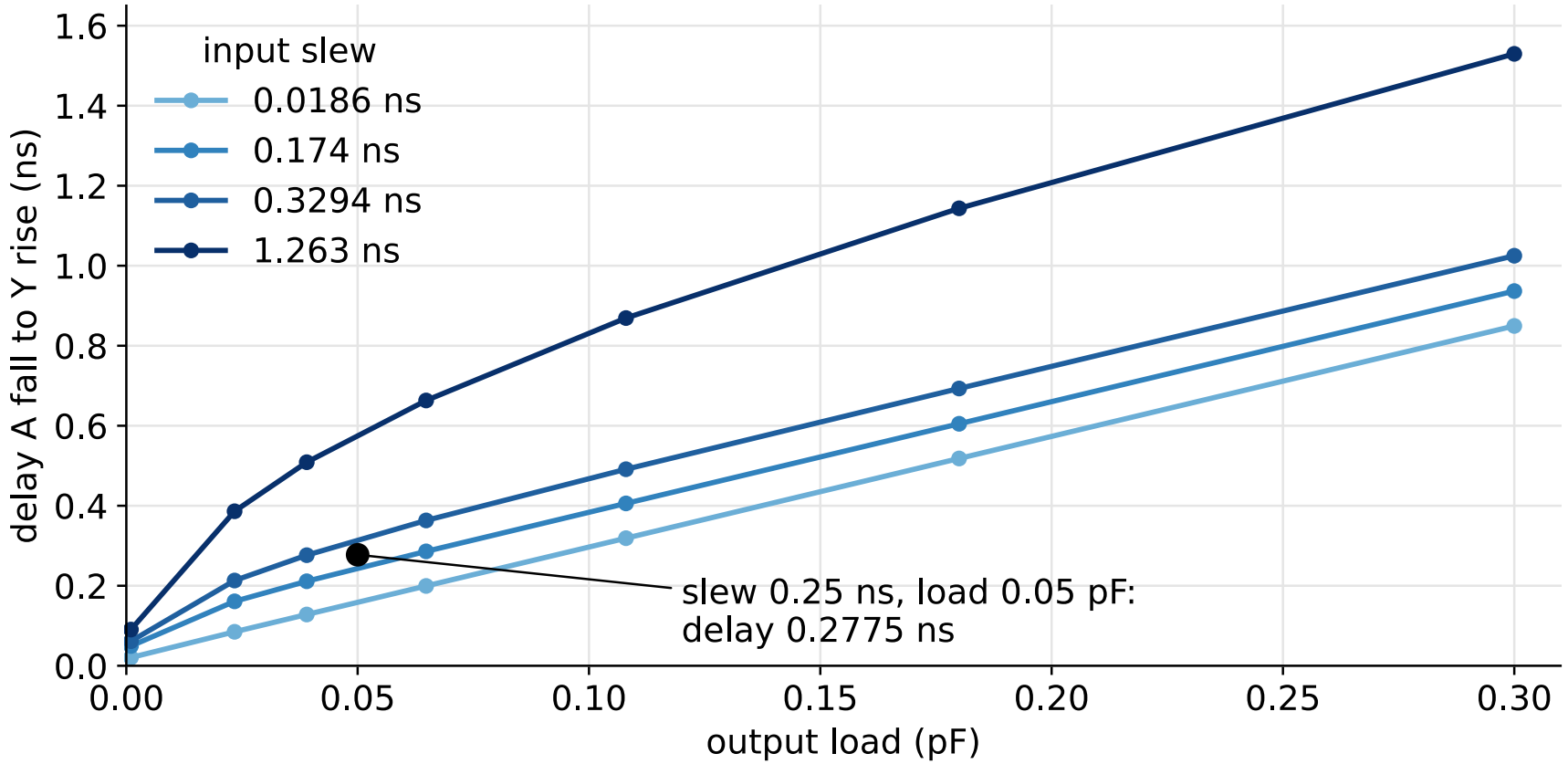
```
ff (IQ,IQN) {
  clear : "!RESET_B";
  clocked_on : "CLK";
  next_state : "D";
}
pin (Q) { function : "IQ"; ... }
```

- `dfrbpq_1`: state `IQ`, takes `D` at the rising `CLK`
- Cleared while `RESET_B` is low
- Setup and hold: **4×4 tables** over data and clock slew
- Every flip-flop of SG13CMOS5L has a **reset**

Delay Calculation with Tables



Nonlinear Delay Model (NLDM)



Fast input: almost **linear** in the load. Slow input: the curve **bends**

Interpolation Between Four Entries

cell_rise	C = 0.039 pF	C = 0.0648 pF
s = 0.174 ns	0.2112 ns	0.2860 ns
s = 0.3294 ns	0.2764 ns	0.3633 ns

- Input slew s = 0.25 ns
- Load C = 0.05 pF

- Along the load: u = (0.05 − 0.039) / (0.0648 − 0.039) = 0.4264
- Along the slew: v = (0.25 − 0.174) / (0.3294 − 0.174) = 0.4891
- Each row: $T_1 + u \cdot (T_2 - T_1)$ gives 0.2431 ns and 0.3135 ns
- $t_{pd} = 0.2431 + v \cdot (0.3135 - 0.2431) =$ **0.2775 ns**

OpenSTA Shows Its Work

```
A v -> Y ^
P = 1.0000 V = 1.2000 T = 25.0000
------- input_net_transition = 0.2500
|       total_output_net_capacitance = 0.0500
|        0.0390     0.0648
v      --------------------
0.1740 | 0.2112     0.2860
0.3294 | 0.2764     0.3633
Table value = 0.2775
PVT scale factor = 1.0000
Delay = 0.2775
```

- `report_dcalc -from u1/A -to u1/Y`
- v falling, ^ rising edge
- Same entries, **same result**
- The output slew comes from `rise_transition`

Slews Propagate, Wires Add Load

- A path is computed **stage by stage**: output slew = next input slew
- Load = input pins of the fanout + the **wire**
- 12 **wire-load models**: estimate from fanout and design size, e.g., `1k`
  - only with `set_wire_load_model`, e.g., `1k`: 0.0182 pF for 8 loads
  - without it: **no wires**, as in all our analyses so far
- LibreLane: wires from the **placement**, then from the **routed layout**

Exercise (Section 5.5): interpolate a delay table by hand and with OpenSTA.

Corners



Process, Voltage, Temperature

Corner	Process	Voltage	Temperature	inv_1 rise delay
fast_1p32V_m40C	fast	1.32 V	−40 °C	0.205 ns
typ_1p20V_25C	typical	1.20 V	25 °C	0.278 ns
slow_1p08V_125C	slow	1.08 V	125 °C	0.376 ns
fast_1p65V_m40C	fast	1.65 V	−40 °C	0.154 ns
typ_1p50V_25C	typical	1.50 V	25 °C	0.208 ns
slow_1p35V_125C	slow	1.35 V	125 °C	0.276 ns

inv_1 at an input slew of 0.25 ns and a load of 0.05 pF

- 1.2 V family: slow about **1.8 times** slower than fast
- Setup usually limited by the **slow**, hold by the **fast** corner, sign-off checks **all**

Synthesis with Yosys in Practice



The Synthesis Script

```
yosys -p "read_verilog -sv rtl/fifo.sv; synth -top fifo; \
  dfflibmap -liberty $LIB; abc -liberty $LIB; opt_clean; \
  stat -liberty $LIB; write_verilog -noattr fifo.nl.v"
```

- `synth`: elaborate, optimize, lower to generic gates
- `dfflibmap`: flip-flops onto library flip-flops
- `abc -liberty`: logic onto cells, with **function, area, delay**
- `stat -liberty`: cells and area, `write_verilog -noattr`: the netlist

Area or Speed?

- `abc -constr abc.constr`: adds **buffering and gate sizing**
  - the file sets the driving cell `inv_2` and an output load of 10 fF
- `-D 1000`: a delay target in ps for the sizing
- `abc -script +strash;dch,-f;amap;topo`: **area-only** mapping
- LibreLane `SYNTH_STRATEGY`: `AREA 0` to `3`, `DELAY 0` to `4`
  - default `AREA 0`, the counter of the template uses `DELAY 4`

Exercise (Section 5.7): area against delay on a 32-bit barrel shifter, in two corners.

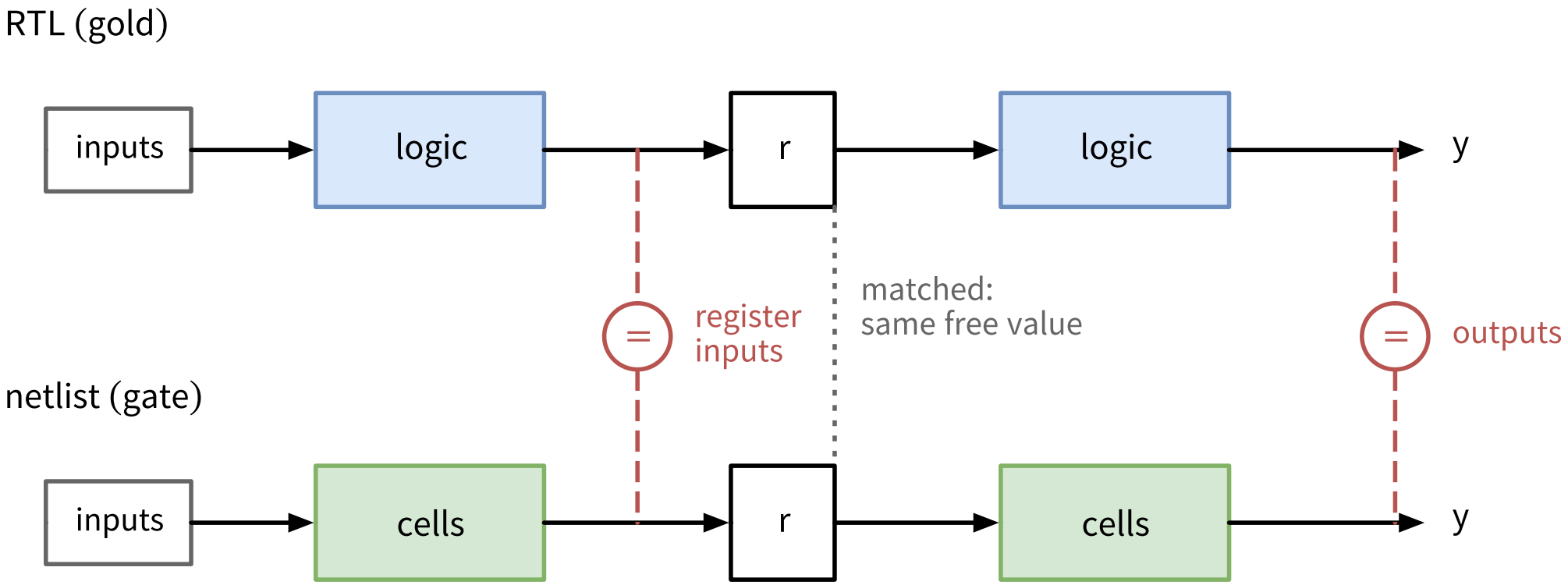
Logic Equivalence Checking



Is the Netlist Still the RTL?

- Synthesis has **bugs**, settings and scripts can change the function
- Netlists are **edited by hand** (ECO) or **stale**
- Gate-level simulation: only the **applied stimuli**
- **Logic equivalence checking** (LEC) proves for **every input**
  - netlist = RTL, without stimuli, without timing

Classic LEC: Cut Points and Cones



- Gold = RTL, gate = netlist: match **registers and outputs** by name
- Every **cone** equal: equivalent **from any state**

Limits of LEC

- Fails falsely when synthesis changes **registers**
  - FSM re-encoding, retiming, merged or duplicated registers
  - then: matching rules, or a **sequential** check (SEC) from reset
- Runs after **every netlist change**: synthesis, place and route, scan, ECO
- Sees no **timing**, and no **start values** of registers without reset
- Proves netlist = RTL, **not that the RTL is correct**

EQY: Gold and Gate

```
[gold]
plugin -i slang
read_slang ../../../verification/code/fifo/rtl/fifo.sv --top fifo
hierarchy -check -top fifo
proc
memory_map
flatten -noscopeinfo
[gate]
read_liberty -ignore_miss_func stdcell.lib
read_verilog fifo.nl.v
hierarchy -check -top fifo
flatten -noscopeinfo
select -assert-none t:sg13cmos5l_*
[collect *]
bind full_o
bind empty_o
[strategy sby]
use sby
engine abc pdr
```

- Gold through **slang**, not the synthesis frontend
- Gate: cell functions from **Liberty**
- `hierarchy -check`, `select -assert-none`: a missing cell model is an **error**
- `bind`: later today

How EQY Proves

- **Partitions**: one per matched register or output
  - signals of **other** partitions are free inputs, its **own registers stay inside**
- Proved **over time** with PDR: equal at the matched signals in **every cycle**
  - netlist registers start **anywhere**, RTL registers start at **x**
  - an **x** on the gold side matches any value
- Equal in every state the RTL **reaches after reset**
  - unreachable states are **not compared**, unlike a classic checker

Live Demo: EQY on the FIFO

```
LIB=$PDK_ROOT/ihp-sg13cmos5l/libs.ref/sg13cmos5l_stdcell/lib
LIB=$LIB/sg13cmos5l_stdcell_typ_1p20V_25C.lib
cd content/synthesis_lib/code/lec
ln -sf $LIB stdcell.lib
yosys -p "read_verilog -sv ../../../verification/code/fifo/rtl/fifo.sv; \
  synth -top fifo; dfflibmap -liberty $LIB; abc -liberty $LIB; \
  opt_clean; write_verilog -noattr fifo.nl.v"
eqy -d fifo_eqy fifo.eqy
```

- `Successfully proved designs equivalent`, `DONE (PASS, rc=0)`, about 3 s
- `[collect]` merged the partitions to **3**, see `fifo_eqy/partition.list`
- Then delete the `collect` section and run again: exit status **2**

A False Failure

- Without `bind`: **5 of 9** partitions fail, e.g., `wr_ptr_q`
- Counterexample after reset: `push_i` = 1, `full_o` = 0, `rd_ptr_q` = 100
  - gold: `wr_ptr_q` 000 → 001, gate: stays 000
- Gold reads the free `full_o`, gate recomputes **full** from the free `rd_ptr_q`
  - wrap bit (Chapter 4): pointers 000 and 100 mean **full**
  - but `full_o` = 0: an **impossible state**
- `bind`: the readers of a flag join the flag's partition

A Check Must Be Able to Fail

- A failing partition is **not yet a bug**: read the counterexample
- A passing check must **fail on a broken design**
  - stale RTL with mutant M3 of Chapter 4: fails in `full_o`
- Other tools: kepler-formal (LEC and SEC), Conformal, Formality
- LibreLane's EQY step prepares cell models **for sky130 only**

Exercise (Section 5.8.2): prove a barrel shifter equivalent, then break its netlist.

Summary

- Synthesis: **elaborate, optimize, lower, map**, only the mapping reads Liberty
- Standard cells: one **height**, whole **sites**, shared **rails**, many **views**
- Liberty: function, area, pin loads, **timing arcs** with **7×7 tables**
- Delays by **interpolation**, slews propagate stage by stage
- **Corners**: setup usually limited by the slow, hold by the fast corner
- Mapping trades **area against delay**: try and compare
- **LEC** proves netlist = RTL, EQY from reset, and must be shown to **fail**

Exercises

1. Interpolate a Delay Table (5.5)
2. Area Against Delay (5.7)
3. Prove It, Break It (5.8.2)

All code in `content/synthesis_lib/code` of the course repository, every number reproduced by `scripts/check_synthesis_lib.sh`



Chapter 5



JOHANNES KEPLER
UNIVERSITY LINZ